

Uvod v evolucijske algoritme

Vaje 2025/2026

4 Projekt: Pametni paketnik – reševanje problema TSP

V okviru projekta boste nadgradili vašo mobilno aplikacijo z evolucijskim algoritmom za reševanje problema **trgovskega potnika (TSP)**. Pri problemu TSP imamo podan seznam mest in razdalj (uteži) med njimi. Pri reševanju problema iščemo najkrajšo možno pot, kjer vsako mesto obiščemo natanko enkrat in se vrnem v izvirno mesto. V vaši aplikaciji boste iskali najkrajšo/najhitrejšo pot med paketniki in jo prikazali na zemljevidu. Nalogo boste reševali v skupinah, pri čemer je razdeljena na dva dela. V prvem delu boste implementirali algoritem in ga testirali na testnih problemih (benchmark). V drugem delu pa boste algoritem zagnali na realnem problemu in ga vključili v mobilno aplikacijo. Ker boste algoritem zaganjali na Android napravi, mora biti implementiran v programskem jeziku Kotlin ali Java.

Za pomoč pri implementaciji si pomagajte s priloženimi datotekami. V evolucijskem algoritmu, pot predstavlja eno možno rešitev/posameznika in dolžina poti predstavlja fitnes. Razred **TSP** predstavlja problem trgovskega potnika, kjer so shranjeni podatki o mestih. V razredu **TSP** morate dopolniti nekaj metod. Metoda **calculateDistance** podpira različne izračune razdalj med mesti. Metoda **generateTour** se uporablja za generiranje novih posameznikov. Posameznik je predstavljen z razredom **Tour** (Solution), ki ga sestavlja zaporedje mest (razred **City**). V metodi **loadData** boste prebrali vse potrebne podatke o problemu. Podatki so shranjeni v datotekah s končnico **.tsp**. Da lahko metoda odpre datoteke, se morajo nahajati v mapi, ki je označena kot **Resources Root**. Datoteke **.tsp** imajo točno določen format, ki je podan v datoteki **tsplib95.pdf**. Podpreti morate razčlenjevanje dveh tipov povezav (angl. edge) med mesti. Tip povezave je določen s ključno besedo **EDGE_WEIGHT_TYPE**. Podprli boste tipa **EXPLICIT** in **EUC_2D**. Pri tipu **EXPLICIT** so v datoteki zapisane uteži med mesti. Uteži so lahko zapisane na različne načine, kar je določeno s ključno besedo **EDGE_WEIGHT_FORMAT**. Podpreti morate vsaj format **FULL_MATRIX**, kjer je podana celotna matrika (spremenljivka **weights** v razredu **TSP**). Pri tipu **EUC_2D** so v datoteki podane koordinate mest. Za izračun razdalje med dvema mestoma se uporabi evklidska razdalja. Primer formata **EXPLICIT** najdete v datoteki **bays29.tsp** in primer formata **EUC_2D** v datoteki **eil101.tsp**.

Dodatne primere datotek **.tsp** lahko najdete tukaj:

<http://www.math.uwaterloo.ca/tsp/vlsi/summary.html>

<https://people.sc.fsu.edu/~jburkardt/datasets/tsp/tsp.html>

<http://elib.zib.de/pub/mp-testdata/tsp/tsplib/tsp/index.html>

<http://comopt.ifl.uni-heidelberg.de/software/TSPLIB95/tsp/>

Ko boste imeli pripravljen razred **TSP**, lahko pričnete z implementacijo evolucijskega algoritma. V evolucijskem algoritmu uporabite elitizem in sledeče genetske operatorje: **tournament selection**, **Partially Mapped Crossover (PMX)** in **swap mutation**. Kjerkoli boste v vaši kodi potrebovali naključje, za to uporabite pripravljen razred **RandomUtils**. Če ste ga pravilno uporabili, bi morali pri istem semenu zmeraj dobiti isto rešitev.

4.1 Testni problemi

Pri testiranju uporabite privzeto seme. Po zaključku preizkusa algoritma na začetku pokličite metodo **setSeedFromTime()**.

Pri eksperimentu uporabite sledeče parametre:

- Število ponovitev: 30
- maxFes: 1000 * d (št. mest)
- Velikost populacije: 100
- Verjetnost križanja = 0.8
- Verjetnost mutacije = 0.1

Algoritem zaženite za sledeče instance problema:

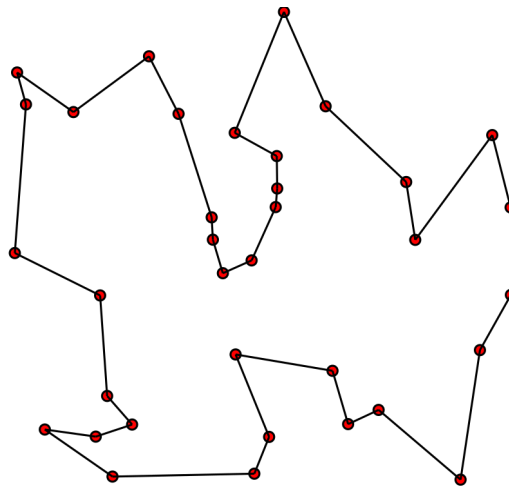
bays29.tsp
eil101.tsp
a280.tsp
pr1002.tsp
dca1389.tsp

Za vsako instanco problema pripravite tekstovno datoteko (**.txt**), ki bo vsebovala najboljše rezultate vseh 30 ponovitev. Format datoteke je enak kakor pri prejšnjih nalogah! Ime datoteke naj bo sestavljeno iz (ime ekipe brez šumnikov)_ime instance problema.txt. Primer imena datoteke za ekipo z imenom OutOfBounds: **OutOfBounds_bays29.txt**. Vso kodo objavljajte sproti na skupen repozitorij skupine in mi dajte dostop (@Ravby). **Rok za oddajo datotek je 3 dni pred zagovorom!**

4.2 Sprotni izris poti (neobvezno)

Dodajte vmesnik, ki bo izrisoval mesta in poti. Poti naj se posodobijo vsakič, ko algoritem najde boljšega posameznika.

Primer izrisa:



Slika 1: Primer izrisa poti za TSP.

4.3 Realni problem

Ko boste stestirali algoritem na testnih problemih, se lotite reševanja realnega problema. V tem sklopu boste iskali najkrajšo pot med vsemi paketniki Direct4.me, ki se nahajajo v datoteki **Direct4me-seznam lokacij.pdf**. Najprej ustvarite matriko razdalj s pomočjo [Distance Matrix API](#)-ja, tako da pokličete API med vsemi kraji. Primer klica lahko najdete [tukaj](#). Pri razvoju si lahko pomagate z obstoječimi [odjemalci](#).

Ustvarite ločeni matriki za razdaljo in čas. Matriki naj bosta tipa **FULL_MATRIX**, pri tem posebej pridobite čas in razdaljo za obe smeri. Pomembno je upoštevati, da sta čas oz. razdalja iz kraja A v B lahko različna kot iz kraja B v A (Asymmetric Traveling Salesman Problem - ATSP). Dodatne informacije o krajih (na primer številko mesta, naslov in koordinate) lahko hranite v ločeni datoteki.

Za pridobitev koordinat krajev uporabite [Geocoding API](#), ki vrne koordinate za podan naslov. Ko bosta matriki pripravljene, lahko začnete z optimizacijo (iskanjem najkrajše poti). Algoritem zaženite večkrat in z različnimi nastavitvami za doseg boljših rezultatov.

Alternativni API-ji za geokodiranje:

- [Geoapify](#)

- [Mapbox](#)
- [OSRM](#)

Pri reševanju realnega problema preizkusite različne parametre algoritma (velikost populacije, verjetnost križanja in verjetnost mutacije), da dosežete čim boljše rezultate!

4.3.1 Vizualizacija

Po uspešni implementaciji problema se lotite integracije knjižnic v vašo aplikacijo Android, kjer boste vizualizirali najdeno rešitev (končno pot) na zemljevidu. Na uporabniškem vmesniku dodajte možnost za izbiro mest ([Recyclerview](#)), ki bodo vključena v iskanje. Privzeto naj bodo označena vsa mesta. Prav tako dodajte možnost za nastavitve parametrov genetskega algoritma (GA) ter izbiro med optimizacijo časa ali dolžine poti. Po zaključku iskanja algoritma naj se prikaže vmesnik z zemljevidom. Na zemljevidu naj bodo poti med kraji izrisane po cestah, ne le kot ravne črte. Poleg tega naj se izpišeta skupna dolžina poti in skupni čas potovanja (odvisno od izbire).

Točkovanje (skupaj 35 točk):

- Implementacija evolucijskega algoritma: **8 točk**
- Testni problemi: **6 točk**
- Sprotni izris poti (**neobvezno**): **3 točke**
- Realni problem: **7 točk**
- Testiranje različnih parametrov (**neobvezno**): **3 točk**
- Integracija v Android in izdelava vmesnika: **8 točk**

* Rok za zagovor: v svojem terminu vaj **med 5. 1. 2026 in 9. 1. 2026**.

* Naloga je **obvezna** in vredna **35 točk** (~20 % če je zagovor teden kasneje).

Za uspešni zagovor je potrebno implementirati vse obvezne točke!